IGNYTION IO · TECHNICAL LAB NOTE

Low-speed USB protocol: From RTL to routed GDS

A beginner's walkthrough of a checked open-source reference build

Follow usb_ls_core from its communication rules and Verilog **source** to a routed IHP SG13G2 layout. Worked calculations, measured digital traces and tool results are identified separately. This is an educational block, not a manufactured chip or a signoff-ready design.

STAGE	WHAT IT PRODUCED	RESULT
Simulation	Listed directed checks	PASS directed
Synthesis	2608 mapped functional cells	PASS mapping
Routing	Routed DEF and extracted SPEF	PASS reference
Timing	TT and slow setup / hold	PASS reference
GDS	Decoded placement agreement	PASS export
Foundry DRC / LVS	No accepted result	UNPERFORMED

CONTENTS

What is in this document

Start with the wire-level rules, then follow the same circuit through the tools. Each section has a specific evidence boundary.

READING MAP

SECTION	PAGE
Start here	p. 03
Chip design in ten minutes	p. 04
Low speed is a separate line state	p. 05
NRZI and a stuffed zero	p. 06
PID integrity and CRC are not optional	p. 07
A ROM-driven restricted HID host	p. 08
What the diagnostic program exercises	p. 09
Integrity gaps remain actionable	p. 10
Reconstruct FF from line states	p. 11
Lint: read code before running it	p. 12
Diagnostic PASS, enumeration unproven	p. 13
Synthesis: a description becomes gates	p. 14
Floorplan and placement	p. 15
Clock tree and routing	p. 16
Two measured reference corners	p. 17
Count the right objects	p. 18
DRC and LVS: what is still missing	p. 19
Everything at a glance	p. 20
Run it yourself in WSL	p. 21
Sources, ownership and remaining work	p. 22
Design and physical glossary	p. 23–24

SECTION 1 · START HERE

Start here

A protocol is an agreement about how connected devices exchange information: which wire carries each bit, when to sample it, and how to recognise a complete transfer.

WHAT WE BUILT AND WHAT IT PROVES

A Verilog description represents circuits, not software instructions running one after another. Registers remember values; combinational logic calculates the next values. A **clock** advances the registered state. The next pages first explain this protocol at its wires, then connect those rules to the actual reference RTL.

The implemented unit is usb_ls_core in hw/protocols/stretch/usb_ls. It is a fixed **reference** block, not the Jane Street programmable combined emulator or a Tiny Tapeout submission top. No new competition architecture, instruction set, pin map or authority decision is implied by these documents.

Receive CRC is discarded and PID matching uses only the low nibble. The simulated diagnostic ROM differs from the hardened production ROM. Later pages give the exact stimulus and the implementation evidence rather than using a broad protocol-compliant label. A successful mapped netlist, routed DEF and decoded **GDS** establish concrete implementation/export results, not readiness to manufacture.

The reading path starts with chip-design vocabulary, then develops the protocol, calculations, local interface and failure cases. Separate sections explain lint, simulation, synthesis, placement, routing, timing and **GDS**. The last two pages define design and physical terms. No prior ASIC-flow experience is assumed.

EVIDENCE VOCABULARY

Label	Meaning in this document
Worked example	Explicit assumptions; independently calculated
Measured trace	Positive digital regression VCD, where present
Directed PASS	Listed assertions and negative controls
Routed / decoded GDS	Physical export and structural agreement
Signoff	Not performed or claimed

SECTION 2 · CHIP DESIGN

Chip design in ten minutes

Each stage makes the same circuit more concrete. The folders below are the actual unit layout, not a copied APB command list.

CHIP DESIGN

RTL states what hardware should do. A simulator checks selected input sequences before any silicon exists. Synthesis replaces that description with gates from a process library. Physical tools choose positions and metal routes. Timing analysis checks the resulting delays; stream-out writes the mask-layout geometry.

THE PIPELINE, STAGE BY STAGE

FOLDER	WHAT HAPPENS HERE
00_lint	Read code for risky patterns; retained warnings
01_rtl	Pinned source and the local wrapper
02_sim	Independent stimulus, comparisons and fatal controls
03_verification	Supplemental checks; not a universal UVM PASS
04_synthesis	Map functional logic to SG13G2 cells
05_dft	Manufacturing test work remains unperformed
06_floorplanning	Die outline, rows, power and pin locations
07_placement	Choose legal positions for mapped cells
09_place_and_route	Clock tree, routing and SPEF extraction
08_sta	Read post-route TT and slow timing reports
10_gds	Stream-out and independent placement decoder
11_physical_verification	Foundry DRC / LVS remain unperformed

SECTION 3 · ELECTRICAL BOUNDARY

Low speed is a separate line state

The restricted host uses a 12 MHz clock and eight clock ticks per low-speed bit.

ELECTRICAL BOUNDARY

A 1.5 Mbit/s line has a 666.667 ns nominal bit interval. The **reference** clock period is 83.333333 ns, so eight ticks occupy one bit. In low-speed signalling the J idle state has D- high and D+ low; K is the opposite differential state. SE0 places both low. These are logical states, not a claim about pad voltages or measured rise/fall times.

The digital wrapper exposes separate input, output and output-enable signals. A real host needs suitable electrical pads or a transceiver, host pull-downs, a protected VBUS supply, connector and grounding. Those elements are absent from this block. The report/debug outputs likewise do not turn it into a general USB controller with an application bus.

The hardened configuration sets FULL_SPEED=0. A full-speed example from another test cannot be reused by changing only a caption. Bit timing, idle polarity and the physical connection role must all agree with the low-speed configuration.

LOW-SPEED LOGICAL STATES

State	D+	D-
J idle	0	1
K	1	0
SE0	0	0
SE1	1	1; not normal packet data

SECTION 4 · LINE CODING

NRZI and a stuffed zero

Data bits are reconstructed from changes between sampled differential states.

LINE CODING

NRZI sends a zero by changing between J and K and a one by retaining the prior state. The byte is still processed least-significant bit first. A run of six consecutive data ones requires insertion of a zero before the next original bit. The inserted bit changes the line state but is removed by the receiver rather than delivered as payload.

For an isolated FF byte, eight original one bits occupy nine encoded slots because one zero is inserted after the sixth one. With a starting J state, the first six ones retain J, the stuffed zero changes to K, and the final two ones retain K. An adjacent byte can alter the stuffing count because the consecutive-one state crosses byte boundaries.

Nine slots at 1.5 Mbit/s take 6 us, compared with 5.333 us for eight unstuffed slots. This classroom calculation omits SYNC, PID, CRC and packet boundaries. A **payload** byte rate cannot be inferred by dividing the line bit rate by eight and ignoring these costs.

WORKED ISOLATED FF

Slots	Original data / line action
1..6	Six ones; hold J
7	Stuffed zero; change to K
8..9	Remaining two ones; hold K
Decoded payload	FF; stuffed slot removed

SECTION 5 · PACKET RULES

PID integrity and CRC are not optional

Line-code success alone does not prove packet acceptance or USB compliance.

PACKET RULES

A USB PID carries a four-bit packet identifier and its complemented high nibble. DATA0 is C3: the low nibble is 3 and the high nibble is C. Requiring the XOR of those nibbles to be F detects an inconsistent PID complement. The imported receive path recognises PIDs from only the low nibble; this additional integrity check is not demonstrated.

A data packet also carries **CRC16** over its payload, using the USB data CRC polynomial. The reflected byte-by-byte classroom calculation starts at FFFF, uses A001 for least-significant-bit-first updates, and complements the final state. For ASCII 123456789 this convention produces B4C8. This is a worked arithmetic vector, not a new passing DUT CRC test.

Source review and the directed bench show the present receive engine discarding its two CRC bytes. The bench deliberately sends 00 00 and says they are ignored. A malformed received CRC can therefore leave the payload path apparently successful. The whitepaper does not label this engine a compliant general-purpose USB implementation.

INTEGRITY BOUNDARY

Check	This reference
NRZI and inserted-zero handling	Directed simulation
PID complemented nibble	Not established; low nibble used
Received payload CRC16	Not validated
Complete USB compliance	Not claimed

SECTION 6 · RTL INTERFACE

A ROM-driven restricted HID host

The wrapper embeds a synchronous 1024-by-4-bit instruction ROM.

RTL INTERFACE

usb_ls_core instantiates usb_hid_host with FULL_SPEED fixed to zero. The default ROM_FILE is the production HID microcode image in 01_rtl. A synchronous ROM read supplies four-bit instructions to the controller. The wrapper exposes device_type, report_valid, connection_error, busy, report_data and hid_registers, alongside the differential pin interface.

Reset is synchronous active high. The cs input remains high for normal execution. Inputs pass through the upstream sampled filter, whose presence is not itself a CDC signoff claim. The 64-bit report/debug values are specific to this implementation and should not be confused with arbitrary bulk-transfer payload ports.

The simulation overrides ROM_FILE with a short diagnostic program. This deliberately exercises the real top-level line-code machinery, but it does not execute the production enumeration program used in the physical build. The distinction is a verification boundary, not merely a different path name.

SOURCE / COMMANDS

```
module usb_ls_core #(
parameter ROM_FILE = "01_rtl/usb_hid_host_rom.mem"
)(
input wire clk, rst, cs,
input wire usb_dm_i, usb_dp_i,
output wire usb_dm_o, usb_dp_o, usb_oe,
```

01_rtl/usb_ls_core.v:10; excerpt, not a complete module

SECTION 7 · CONTROL SEQUENCE

What the diagnostic program exercises

The diagnostic ROM is a controlled stimulus path, not an enumeration claim.

CONTROL SEQUENCE

The bench keeps cs asserted, resets the controller, then decodes its transmitted stream independently. Expected bytes are 80, C3, FF, 00, A5, AA and BB. The sequence includes a synchronisation byte, a data PID and data patterns chosen to exercise both line transitions and long runs of ones. AA and BB are expected diagnostic bytes, not evidence of valid generated CRC for the preceding packet.

The receive model drives a separate stuffed **NRZI** stream. Its first payload byte is FF; subsequent bytes are formed by (index x 37) XOR A5 and truncated to eight bits. The receive monitor compares each reported byte with this independent expectation. A timeout catches a stalled program rather than allowing an endless wait to resemble a quiet pass.

After packet reception, the test asserts **reset** and checks released line/device state. Production HID discovery, descriptors, endpoint scheduling, recovery across real devices and all control-transfer phases remain outside this diagnostic run.

INDEPENDENT DIAGNOSTIC EXPECTATIONS

Direction	Expected content
TX byte stream	80 C3 FF 00 A5 AA BB
RX payload	FF; then (k x 37) XOR A5
RX CRC bytes	00 00; knowingly ignored
Reset result	Output released; device state quiescent

SECTION 8 · FAILURE ANALYSIS

Integrity gaps remain actionable

A known limitation should determine the next verification task, not disappear behind a PASS label.

FAILURE ANALYSIS

A receiver that ignores CRC can report a corrupted **payload** as if it were usable. A PID matcher that checks only the low nibble can classify some invalid identifiers as recognised packet types. These are source-backed limitations of the imported engine. The directed line-code test cannot repair either contract simply by producing correct example packets.

The production microcode is retained and hardened, but its real-device enumeration behaviour is unverified here. A follow-up must first define expected descriptor exchanges and error responses, then use decisive checks against the production ROM. Adding more diagnostic **payload** bytes would improve data-pattern coverage but would not close that different obligation.

Physical host circuitry also remains necessary. VBUS protection, disconnect/connect detection, pad tolerances and board captures cannot be inferred from a digital **GDS** file. Any deployment decision must treat this as a restricted educational reference with explicit missing guarantees.

NEXT VERIFICATION OBLIGATIONS

Area	Needed evidence
Packet integrity	Bad PID / CRC rejected decisively
Production ROM	Enumeration scenarios and faults
Electrical host	Pads, VBUS, connector and measured timing
Compliance	Applicable USB test plan, not this bench

SECTION 9 · WORKED EXAMPLE

Reconstruct FF from line states

This page uses an ideal logical example; no USB waveform file is claimed.

WORKED EXAMPLE

Beginning in J, the isolated data sequence 1 1 1 1 1 1 1 1 is encoded as six holds, one inserted transition, then two holds. A decoder comparing adjacent line samples obtains six ones, recognises the mandatory zero after that run, removes it, and continues with the remaining two ones. The recovered byte is FF.

EOP in the directed low-speed test is two SE0 bit intervals followed by J. At eight clocks per bit, two SE0 intervals occupy sixteen 12 MHz **clock** ticks, or 1.333333 us. The bench checks that boundary independently of its payload-byte comparisons. An EOP duration check is still not an analogue waveform-compliance measurement.

The USB bench currently retains assertions and logs rather than a plotted positive VCD. Accordingly this document offers a labelled ideal table and reproducing **source**, not a fabricated measured chart. Capturing and decoding a production-ROM waveform is useful follow-up work.

IDEAL SAMPLE DECODER

Encoded observation	Recovered interpretation
J J J J J J	Six consecutive ones
K after six ones	Stuffed zero; remove
K K	Final two ones
SE0 SE0, then J	Directed EOP boundary

SECTION 10 · LINT

Lint: read code before running it

Verilator checks legal but suspicious code patterns without exercising transactions. A warning count is a review starting point, not proof of correctness.

LINT

The retained report contains 26 warning messages and 0 error messages. Categories present: DECLFILENAME, PINMISSING, PROCASSINIT, UNUSEDPARAM, UNUSEDSIGNAL. These figures count report message headers; repeated warnings are not necessarily independent defects. The report is hashed in this document evidence manifest.

The invocation deliberately uses -Wno-fatal so warnings remain visible while lint execution can complete. A successful command therefore does not mean every warning was resolved. Width changes, incomplete cases, **reset** use and unconnected ports need review against the wrapper contract and directed tests.

Lint cannot show that a receive byte is correct, that a transaction finishes, or that an external electrical interface is safe. Those questions require a checking testbench, implementation evidence and separate board or signoff work.

RETAINED WARNING REVIEW

MEASURE	VALUE
Warning message headers	26
Error message headers	0
Clean warning-free RTL?	Not claimed
Protocol behaviour proved by lint?	No

SOURCE / COMMANDS

```
make lint
# inspect, do not just count
# 00_lint/lint_report.txt
```

00_lint/Makefile and run_flow.sh: retained Verilator report

SECTION 11 · SIMULATION AND VERIFICATION

Diagnostic PASS, enumeration unproven

A testbench is code that drives the circuit and judges its outputs. These directed checks use independent expected data and deliberately failing controls.

SIMULATION AND VERIFICATION

The passing diagnostic test checks independent TX and RX **NRZI**, stuffing, two-bit SE0 EOP, eight payload bytes and reset. A deliberately changed expected receive byte produces a fatal failure. This gives stronger evidence than a run that merely reaches finish, while keeping the stimulus narrow and reproducible.

The imported upstream cocotb example is preserved but not accepted as decisive verification: its assertions and line timing do not establish the requested low-speed behaviour. The locally checked bench drives the same real top-level core through a diagnostic ROM and uses independent expectations.

Neither the normal nor the negative-control run demonstrates receive CRC checking, a complemented PID check, complete enumeration, multiple HID devices, disconnect recovery or electrical compliance. In particular the production-ROM physical result and diagnostic-ROM functional result must be reported side by side, not merged into a production-functional claim.

PROOF AND DELIBERATE BOUNDARY

Result	Meaning
Diagnostic normal run	Directed line-code / payload PASS
Injected mismatch	Fatal checker rejection
Production ROM in synthesis	Implemented; not enumerated by this test
Foundry / USB compliance	Unperformed

A checker must be able to fail

Completion alone is insufficient. Read the successful log and the expected fatal rejection together; neither is a protocol-compliance certificate.

SECTION 12 · SYNTHESIS

Synthesis: a description becomes gates

A netlist is a list of logic cells and their connections. Yosys translates the RTL into cells from the IHP SG13G2 library.

SYNTHESIS

A process design kit, or PDK, contains the process-specific files used by implementation tools. This **reference** uses IHP SG13G2, not the SKY130 library used by the golden APB primer. Standard cells are predesigned gates and storage elements with characterised timing and physical views.

The **source** is compiled and checked under the unit workspace before synthesis. Yosys maps the functional logic into the pinned IHP SG13G2 standard-cell library. The current netlist contains 2608 functional cells, excluding $scopeinfo metadata. The structural checker rejects remaining generic logic, latches or unresolved instances.

The mapped count excludes $scopeinfo metadata. It is not the count of every placed instance after **clock** buffers, repair cells and fillers are added. It also does not mean that the original protocol has been independently proven for all possible transactions.

MAPPING RESULT

MEASURE	CURRENT RESULT
Top module	usb_ls_core
Mapped functional cells	2608
Technology	IHP SG13G2
Unresolved generic logic / latches	Rejected by structural checker

SOURCE / COMMANDS

```
make synth
node verify_build.mjs synth
```

04_synthesis/synth.tcl; verify_build.mjs synthesis checks

SECTION 13 · PHYSICAL IMPLEMENTATION

Floorplan and placement

The floorplan gives the circuit a physical outline. Placement chooses where its cells sit inside that outline.

PHYSICAL IMPLEMENTATION

The die outline for this standalone block is 500 x 500 um. The floorplan establishes rows and **reference** power/pin geometry. The placer must fit cells legally without overlaps and with room for routing. This is a block-level educational layout, not a complete padframe or package design.

Mapped functional logic has 2608 cells. After physical additions and fillers, the routed DEF contains 49,699 placed standard-cell instances. A filler occupies empty row sites; it is not another functional logic gate. That distinction explains why the two counts differ so much.

Utilisation is the fraction of core area occupied by placed cell area. It is not simply functional-cell count divided by die size, and it changes between stages. This note does not substitute APB utilisation, row count or I/O count for unavailable protocol-specific measurements.

ACTUAL PHYSICAL REFERENCE

MEASURE	VALUE
Die outline	500 x 500 um
Mapped functional cells	2608
Placed standard-cell instances	49,699
Physical inputs	06_floorplanning; 07_placement
Placement agreement	DEF compared with decoded GDS

SECTION 14 · CLOCK AND WIRES

Clock tree and routing

Clock-tree synthesis distributes clocks to registers. Routing draws the metal connections between placed cells.

CLOCK AND WIRES

A register samples its input at a **clock** edge. Clock buffers distribute that edge across the block; the difference in arrival times is called skew. The reference is constrained at 12 MHz / 83.333333 ns.

OpenROAD completes the staged physical flow. The **reference** die is 500 x 500 um. DEF records routed design geometry and cell placement; SPEF captures extracted parasitics used by the timing analysis. The scripts retain stage-specific output so the result can be investigated instead of relying on a final banner.

Real interconnect has resistance and capacitance, called parasitics. Extraction writes these in **SPEF** for post-route timing. The retained router-rule DRC file is empty. That checks the router view of its work; it is not a full foundry-deck check of the exported polygons.

READ THE PHYSICAL EVIDENCE

ARTIFACT	QUESTION IT ANSWERS
09_place_and_route/route.def	Where are cells and routed connections?
09_place_and_route/route.spef	Which parasitics were extracted?
Router-rule DRC report	Any reported routing-rule violations?
08_sta/timing_report.txt	What does extracted reference timing report?

Two checks can both be called DRC

The empty router report is accepted routing evidence. Foundry DRC is a separate unperformed obligation; one result cannot stand in for the other.

SECTION 15 · REFERENCE TIMING

Two measured reference corners

Static timing analysis adds cell and wire delays without simulating input patterns. Setup asks whether data arrives early enough; hold asks whether it stays stable long enough.

REFERENCE TIMING

The physical **clock** constraint is 12 MHz / 83.333333 ns. Post-route timing uses the extracted design rather than an ideal pre-route wire estimate. The report at each corner must be inspected for worst setup and hold slack and for VIOLATED markers. The reported values are not a timing guarantee under unstated board, voltage or temperature conditions.

Current TT setup/hold slack is +76.96 / +0.16 ns. The slow-corner values are +73.83 / +0.21 ns. Both reports contain no VIOLATED marker. These numbers are taken directly from the local report files at document build time; the evidence manifest retains their hashes.

I/O delays are **reference** budgets, not a contract with an external target, transceiver or PHY. Bus synchronisation, reset recovery/removal, all additional corners and modes, signal integrity and board timing need their own analysis. Positive slack in these two reports is therefore called reference timing closure, not foundry or system signoff.

LIVE EXTRACTED SLACK, NS

Corner	Worst setup	Worst hold
TT	76.96	0.16
Slow	73.83	0.21
Status	PASS reference reports	No VIOLATED markers

Positive slack has a stated boundary

Slack is remaining timing margin. These values apply to the listed clocks, I/O budgets and two library corners; they are not a board-timing or manufacturing signoff guarantee.

SECTION 16 · GDS STREAM-OUT

Count the right objects

GDS stores layout polygons and hierarchical cell references. Stream-out combines routed geometry with the real standard-cell layouts from the PDK.

GDS STREAM-OUT

The **GDS** file contains 4,371,854 bytes and 49 structures. The independent decoder finds 49,699 placed standard-cell instances including fillers. That placement count is intentionally much larger than the 2608 mapped functional cells: filler instances occupy unused placement sites and structures are unique hierarchy definitions.

The decoder compares exported cell identities, coordinates and orientations against the routed DEF. A deliberately altered placement/export expectation fails, including under Python optimisation, so acceptance is not implemented with removable assert-only checks. Raw file SHA256 is a run-specific fingerprint because stream-out timestamps can change.

The OpenROAD router-rule DRC report is empty. It is not a foundry-deck DRC result. LVS, antenna, scan/DFT, power integrity, padframe, tap/endcap strategy and manufacturing signoff remain unperformed. No empty routing report is promoted into a DRC-clean tapeout claim.

LIVE EXPORT MEASUREMENTS

Quantity	Current value
Mapped functional cells	2608
GDS bytes	4,371,854
GDS structures	49
Placed standard-cell instances	49,699
Die	500 x 500 um

SECTION 17 · UNPERFORMED SIGNOFF

DRC and LVS: what is still missing

Design rule checking asks whether geometry follows the process rules. Layout versus schematic asks whether the extracted layout circuit matches the intended netlist.

UNPERFORMED SIGNOFF

Foundry DRC requires the applicable, qualified rule deck and an inspected report on the final polygons. Typical checks include minimum widths, spacing and enclosure. No foundry-deck DRC result has been accepted for this protocol **reference**. The router report is narrower evidence.

LVS requires extracting a circuit from the layout and comparing devices and connectivity with the intended circuit. A structural **GDS** decoder checking cell references, coordinates and orientations does not perform that extraction or connectivity comparison. No accepted LVS match is available.

Antenna, scan/DFT, power integrity, padframe, tap/endcap strategy and manufacturing release remain unperformed. The golden APB primer reports its own separate checks; those results do not transfer to SG13G2 or to this design.

SEPARATE ACCEPTED EXPORT FROM MISSING SIGNOFF

CHECK	THIS REFERENCE
Router-rule report	Empty; narrower routing evidence
Decoded GDS placement	PASS structural/export agreement
Foundry-deck DRC	UNPERFORMED
Extracted-layout LVS	UNPERFORMED
Manufacturing readiness	Not claimed

> ### A GDS file is not tapeout approval
>
> This package is suitable for studying and reproducing a block implementation. Do not hand it off as a signoff-clean manufacturing design.

SECTION 18 · RESULTS SUMMARY

Everything at a glance

Read each result together with the question it actually answers. The same PASS label never implies every protocol or manufacturing obligation is closed.

RESULTS SUMMARY

The directed functional result and the physical export result are both useful, but they prove different things. The physical build uses the fixed wrapper configuration and **reference** constraints. Additional modes, external peripherals, arbitrary clock relationships and board operation require their own evidence.

Important protocol boundary: Receive CRC is discarded and PID matching uses only the low nibble. The simulated diagnostic ROM differs from the hardened production ROM.

FULL REFERENCE FLOW

STAGE	TOOL / EVIDENCE	STATUS
Lint	Verilator; 26 retained warnings	Review
Simulation	Independent directed expectations	PASS directed
Negative controls	Deliberate fatal mismatches	PASS rejection
Synthesis	Yosys; 2608 mapped cells	PASS mapping
Floorplan / placement	OpenROAD; 500 x 500 um	PASS reference
Routing / extraction	DEF and SPEF retained	PASS reference
Timing	TT and slow setup / hold	PASS reference
GDS	KLayout; 4,371,854 bytes	PASS export
Foundry DRC / LVS	No accepted report	UNPERFORMED
Full-chip / board	No accepted integration proof	UNPERFORMED

SECTION 19 · RUN IT YOURSELF

Run it yourself in WSL

Use the pinned source, the unit tool lock and the external PDK revision together.

RUN IT YOURSELF

The proven entry point is a WSL login Bash shell. igny init prepares the packaging runtime. Unsetting CRUCIBLE_ENV after initialisation allows the unit crucible.toml workspace selection to control tool resolution rather than an inherited environment override. This is the observed workflow, not a change to Crucible internals.

Each unit has its existing environment and lock. The common tool set includes Icarus, Verilator, Yosys, OpenROAD and klayout-stream. The separate PDK checkout is IHP SG13G2 revision 22f43352dd8219f9007eb659e422e0d5fe28c5fb. A tool lock does not pin that external PDK by itself; IHP_PDK_ROOT must point to the matching installation.

The documented machine could not use the pip klayout package as the expected stream-out executable; igny tool install klayout-stream supplies the proven converter. make lint sim checks functional behaviour, make flow executes the staged implementation, and node verify_build.mjs all inspects artifacts. Review normal and intentionally failing logs before accepting the result.

Re-running a simulation can replace its VCD and invalidate an earlier document trace hash. Retain a document evidence manifest or regenerate the measured figures after the positive run. Do not silently preserve an old chart while claiming it is the newly generated waveform.

SOURCE / COMMANDS

```
igny init
unset CRUCIBLE_ENV
cd hw/protocols/stretch/usb_ls
igny workspace sync
igny tool install klayout-stream
make lint sim
make flow
node verify_build.mjs all
```

Run after entering the repository in WSL login Bash; PDK required.

SECTION 20 · SOURCES AND REMAINING WORK

Sources, ownership and remaining work

The pinned revision identifies the baseline; local integration changes are described separately.

SOURCES AND REMAINING WORK

Upstream: https://github.com/m1nl/usb_hid_host
Pinned commit: e49217661b275554807760d4e12097b0f189316a
Licence: Apache-2.0; original authors retained. Original **source** notices remain in the RTL. The document is authored by Ignytion; this does not transfer ownership of upstream code or remove its licence obligations.

Local modifications, directed test stimulus and **source** boundaries are recorded in hw/protocols/stretch/usb_ls/README.md and the repository append-only .context.md. The first three protocol lessons have separate editable 16:9 student decks; this A4 technical note is a reading and evidence document, not a replacement for their instructor notes.

Primary reading: USB 2.0 specification, chapters 7 and 8, in specs/USB_low_speed.pdf; usb.org is the canonical download.
Canonical specification download: https://www.usb.org/document-library/usb-20-specification

Open work remains explicit: Receive CRC is discarded and PID matching uses only the low nibble. The simulated diagnostic ROM differs from the hardened production ROM. Full-chip integration, manufacturing checks and board validation are not closed by the standalone **reference** flow. The reader can reproduce the listed evidence without assuming those larger obligations have already been completed.

SECTION 21 · DESIGN VOCABULARY

Glossary: design and protocol

Definitions for reading the protocol rules and the checked RTL. Physical implementation terms follow on the next page.

DESIGN AND VERIFICATION

TERM	WHAT IT MEANS
Protocol	Rules for exchanging information between devices
RTL	Registers and the logic moving values between them
Verilog	A language used to describe hardware
FSM	A circuit progressing through defined states
Lint	Static source checks; no transaction execution
Testbench	Code that supplies inputs and checks outputs
Coverage	Which defined situations were exercised
Negative control	An intentionally wrong case the checker must reject
VCD	Digital signal-change trace from a simulation
Handshake	Signals agreeing when data is accepted
NRZI	A zero changes line state; a one retains it
J / K	Low-speed idle / opposite differential states
SE0	Both signal wires low
HID	Human Interface Device class
PID	Packet identifier with a complemented high nibble

SECTION 22 · PHYSICAL VOCABULARY

Glossary: implementation and signoff

These definitions help distinguish a successfully exported reference layout from a manufacturing-ready design.

PHYSICAL IMPLEMENTATION

TERM	WHAT IT MEANS
Synthesis	Translate RTL into a technology-cell circuit
Netlist	Cells and the connections between them
Standard cell	Predesigned logic/storage cell in a process library
PDK	Process design kit: process-specific implementation files
SG13G2	IHP process used by these reference builds
Floorplan	Die/core outline, rows, power and pin geometry
Placement	Choose legal positions for cells
Filler	Physical row-site cell, not functional logic
CTS / skew	Clock distribution / difference in arrival times
Routing / via	Metal connections / links between metal layers
Parasitics / SPEF	Wire resistance/capacitance / extracted data format
Corner	A characterised process, voltage and temperature case
STA / slack	Static timing analysis / remaining timing margin
DEF / GDS	Placement/routing exchange / polygon hierarchy
DRC	Check geometry against applicable process rules
LVS	Compare the extracted layout circuit with its netlist
Signoff	Accepted applicable release checks; unperformed here